

Sprawozdanie z realizacji zadania: Rozwiązywanie układanki 15 za pomocą algorytmów A* i IDA*

1. Opis heurystyk

W programie zaimplementowano dwie heurystyki:

1. Liczba źle ustawionych kafelków (misplaced tiles)

Heurystyka ta liczy, ile kafelków znajduje się na niewłaściwych pozycjach w stosunku do stanu docelowego. Jest to prosta i szybka heurystyka, ale mniej dokładna.

2. Odległość Manhattan

Heurystyka ta sumuje odległości Manhattan (suma różnic współrzędnych w poziomie i pionie) każdego kafelka od jego pozycji docelowej. Jest bardziej dokładna niż liczba źle ustawionych kafelków, ale wymaga większej liczby obliczeń.

Obie heurystyki są dopuszczalne (nigdy nie przeszacowują kosztu dojścia do celu) i monotoniczne (spełniają nierówność trójkąta), co gwarantuje poprawność działania algorytmu A*.

2. Opis generowania permutacji

Permutacje generowane są w dwóch trybach:

1. Losowe ruchy od stanu docelowego

W przypadku podania liczby ruchów k , program wykonuje k losowych przesunięć kafelków od stanu docelowego. Dzięki temu generowane permutacje są zawsze rozwiązywalne.

2. Pełna losowość

W przypadku pełnej losowości program generuje losową permutację kafelków i sprawdza jej rozwiązywalność za pomocą funkcji `isSolvable`. Jeśli permutacja jest nierozwiązywalna, proces jest powtarzany.

3. Opis działania algorytmu

Graf połączeń

Graf stanów układanki jest reprezentowany jako zbiór węzłów, gdzie każdy węzeł odpowiada jednemu układowi kafelków. Krawędzie między węzłami reprezentują możliwe przesunięcia kafelków.

Algorytmy przeszukiwania

1. A*

Algorytm A* korzysta z kolejki priorytetowej, w której stany są sortowane według funkcji kosztu $f(n) = g(n) + h(n)$, gdzie:

- $g(n)$ to koszt dojścia do stanu n ,
- $h(n)$ to wartość heurystyki dla stanu n .

Algorytm eksploruje stany w kolejności rosnącej wartości $f(n)$.

2. IDA*

Algorytm IDA* wykorzystuje iteracyjne pogłębianie z ograniczeniem na wartość $f(n)$. W każdej iteracji zwiększa próg, jeśli nie uda się znaleźć rozwiązania w bieżącym ograniczeniu.

4. Złożoność obliczeniowa

1. Czasowa

- Generowanie sąsiadów: $O(n^2)$ dla planszy o wymiarach $n \times n$.
- Obliczanie heurystyki:
 - Liczba źle ustawionych kafelków: $O(n^2)$,
 - Odległość Manhattan: $O(n^2)$.
- A*: Złożoność w najgorszym przypadku wynosi $O(b^d)$, gdzie b to współczynnik rozgałęzienia, a d to głębokość rozwiązania.
- IDA*: Złożoność podobna do A*, ale z mniejszym zużyciem pamięci.

2. Pamięciowa

- A*: $O(b^d)$ (przechowywanie wszystkich odwiedzonych stanów w pamięci).
- IDA*: $O(d)$ (przechowywanie jedynie bieżącej ścieżki).

5. Mechanizmy przyspieszające działanie

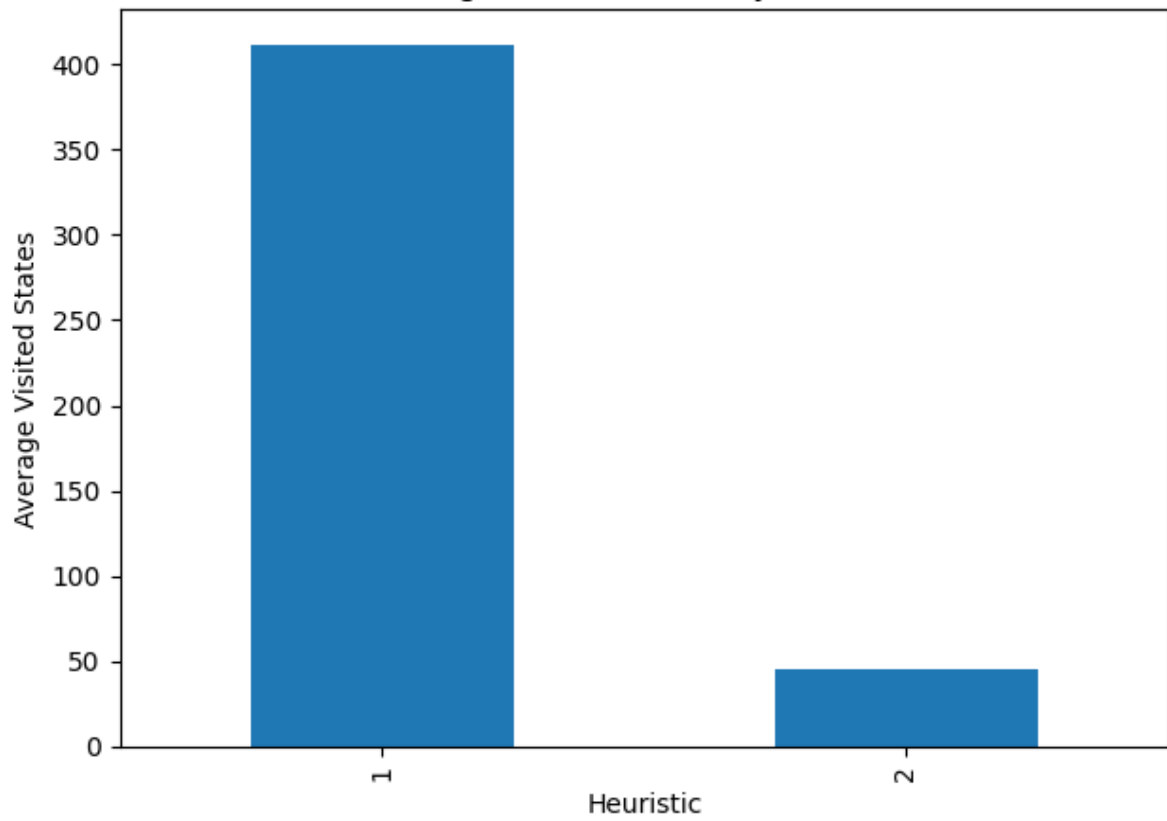
• Mapa odwiedzonych stanów

W algorytmie A* zastosowano mapę (visitedMap), która przechowuje najlepszy koszt dojścia do każdego stanu. Dzięki temu unika się ponownego przetwarzania stanów o wyższym koszcie.

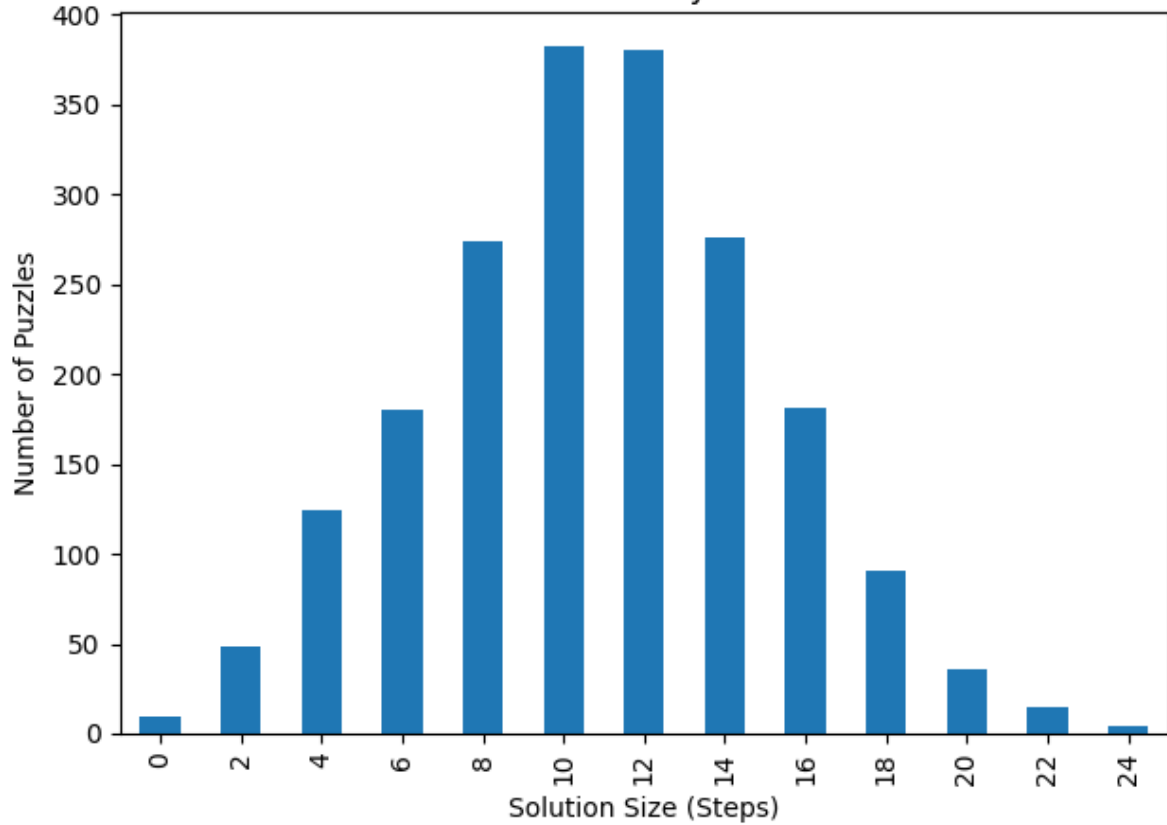
- **Iteracyjne pogłębianie w IDA***

Algorytm IDA* ogranicza pamięć, przeszukując graf w sposób podobny do DFS, ale z ograniczeniem na wartość funkcji $f(n)$.

Average Visited States by Heuristic



Puzzle Distribution by Solution Size



Visited States vs. Solution Size

